

Google Cloud Security Engineering Lab

Part 3 – Cloud Monitoring, Logging & Alerting

Prepared By

Senthamizhan Mathew J

Google Cloud Security Engineering Portfolio

Technologies Used

- Google Cloud Platform (GCP)
- Google Kubernetes Engine (GKE)
- Kubernetes
- Docker
- Artifact Registry
- Cloud Monitoring
- Cloud Logging
- Log-Based Metrics
- Alert Policies
- Notification Channels

Table of Contents

1. Introduction
2. Objectives
3. Architecture Diagram
4. UML Architecture Diagram
5. Project Workflow
6. Environment Setup
7. Implementation Steps
8. Screenshots
9. Results
10. Challenges Faced
11. Learning Outcomes

12. Conclusion

1. Introduction

Cloud Monitoring and Cloud Logging are essential Google Cloud services used to monitor infrastructure, applications, and Kubernetes workloads. This project demonstrates deploying a containerized Flask application on Google Kubernetes Engine, storing Docker images in Artifact Registry, collecting logs, creating log-based metrics, and configuring automated alert policies for proactive monitoring.

2. Objectives

The objectives of this project are:

- Deploy a Kubernetes cluster.
- Create an Artifact Registry repository.
- Push Docker images.
- Deploy a containerized application.
- Configure Cloud Logging.
- Create Log-Based Metrics.
- Configure Alert Policies.
- Configure Notification Channels.
- Monitor workloads.
- Validate monitoring and alerting functionality.

3. Professional Architecture Diagram

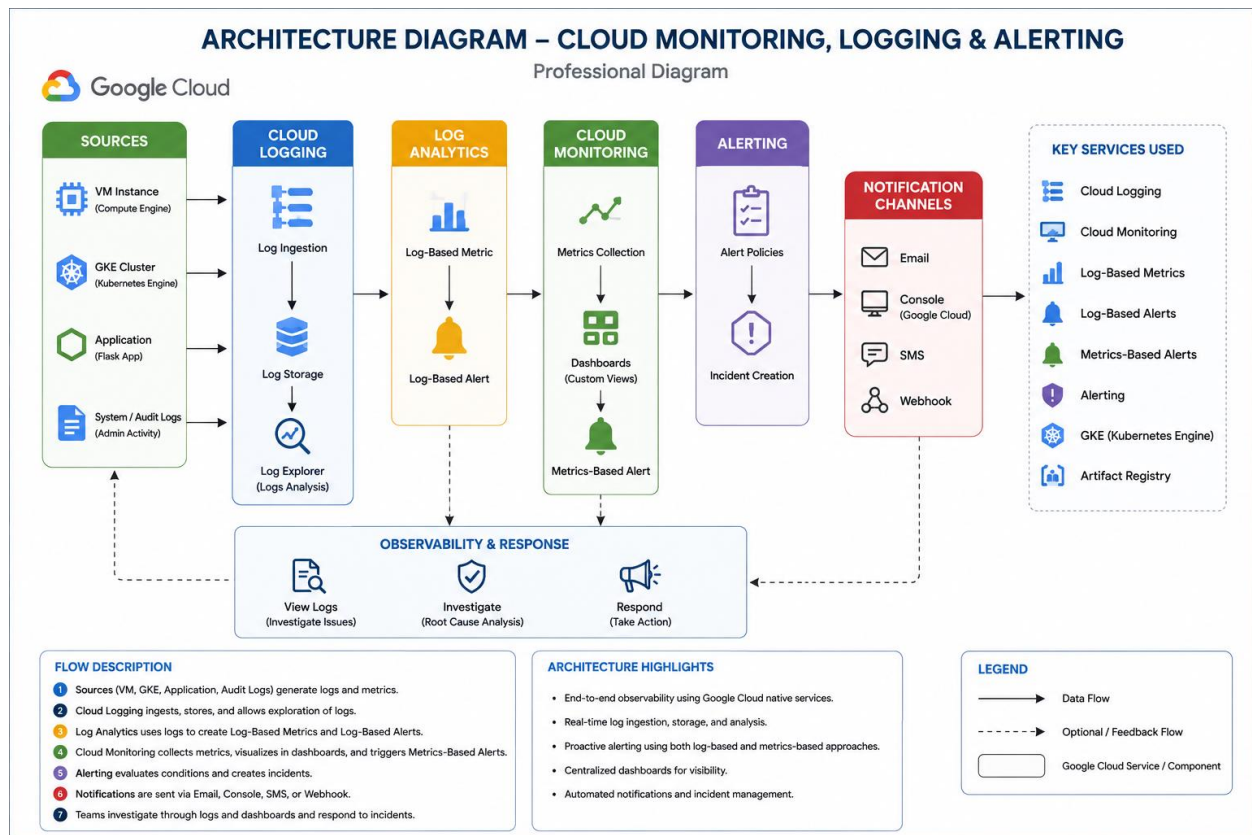


Figure 3.1 – Google Cloud Monitoring Architecture

4. UML Architecture Diagram

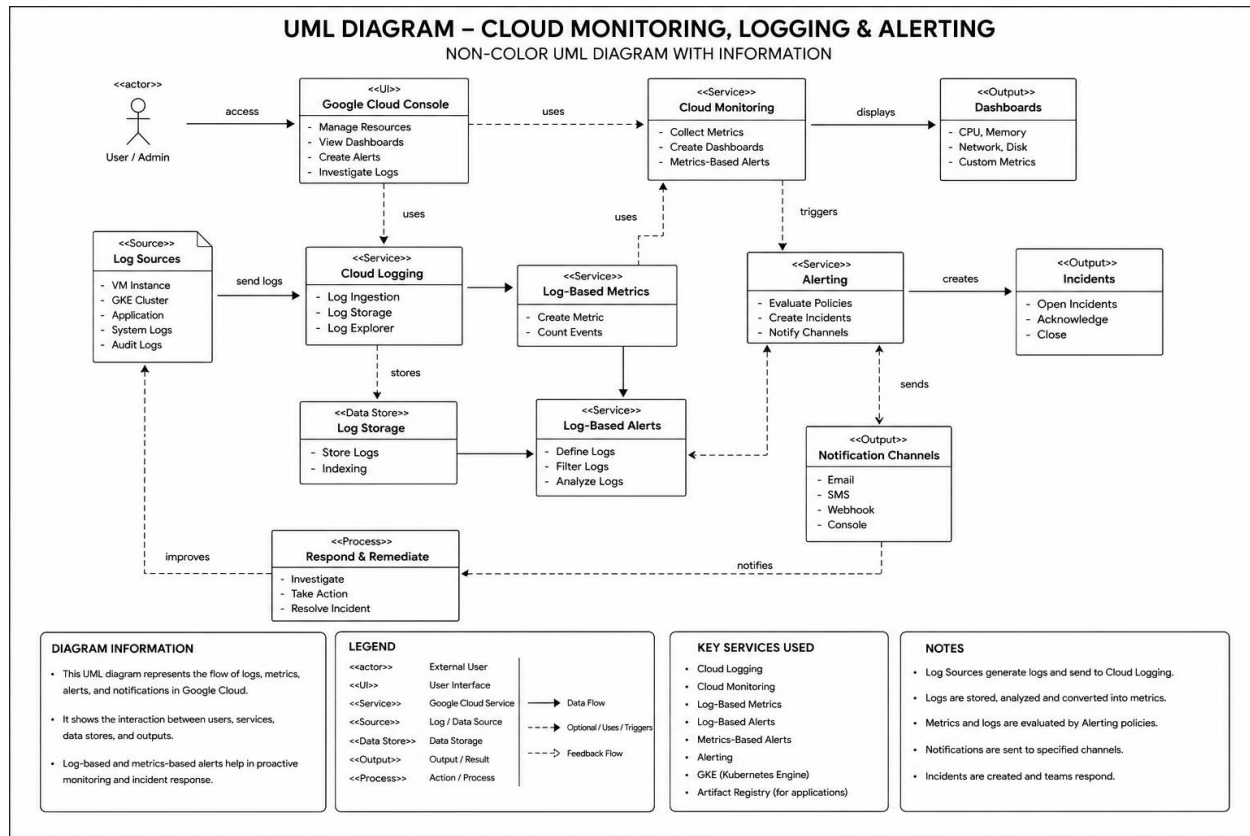


Figure 4.1 – UML Deployment Architecture

5. Project Workflow

The implementation follows the workflow below:

1. Configure Google Cloud environment.
2. Create Google Kubernetes Engine cluster.
3. Create Artifact Registry repository.
4. Upload Docker container image.
5. Deploy Flask application.
6. Expose application using LoadBalancer.
7. Generate application logs.
8. Create Log-Based Metric.
9. Configure Alert Policy.
10. Configure Notification Channel.
11. Validate alerts.

12. Monitor application health.

6. Environment Setup

Cloud Components

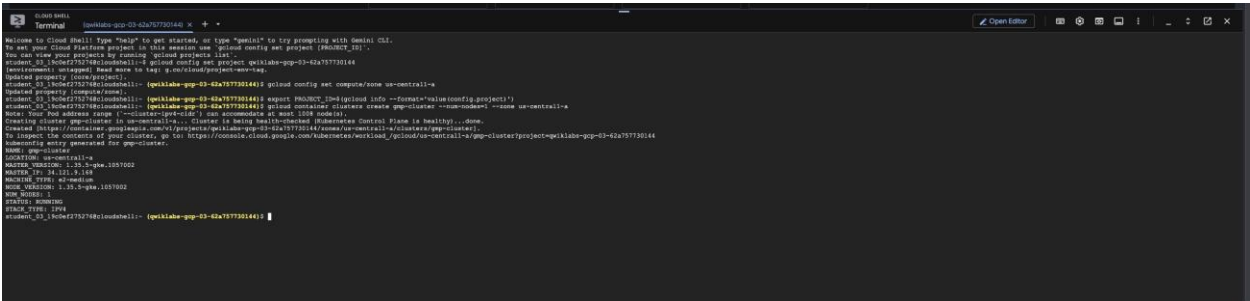
Component	Description
Cloud Platform	Google Cloud Platform
Compute Service	Google Kubernetes Engine
Registry	Artifact Registry
Monitoring	Cloud Monitoring
Logging	Cloud Logging
Container Platform	Kubernetes
Container Runtime	Docker

7. Implementation

Step 1 – Create GKE Cluster

Description

A Kubernetes cluster was created to host the monitoring application.



Step 2 – Verify Cluster Creation

Description

The cluster status was verified using the Google Cloud Console and Cloud Shell.

Step 5 – Configure Deployment YAML

Description

Deployment manifests were updated with the Artifact Registry image.



```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: helloworld-gke
spec:
  replicas: 1
  selector:
    matchLabels:
      app: helloworld
  template:
    metadata:
      labels:
        app: helloworld
    spec:
      containers:
        - name: helloworld
          image: gcr.io/artifactregistry-140412/helloworld:1.0.0
          ports:
            - containerPort: 8080
          env:
            - name: PORT
              value: "8080"
```

Step 6 – Deploy Application

Description

The Kubernetes deployment was created successfully.

Step 7 – Create Kubernetes Service

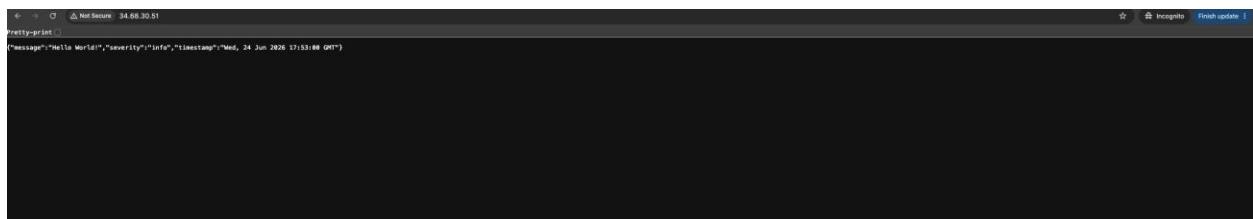
Description

A LoadBalancer service exposed the application to external users.

Step 8 – Verify Running Application

Description

The application was accessed successfully using the external IP.



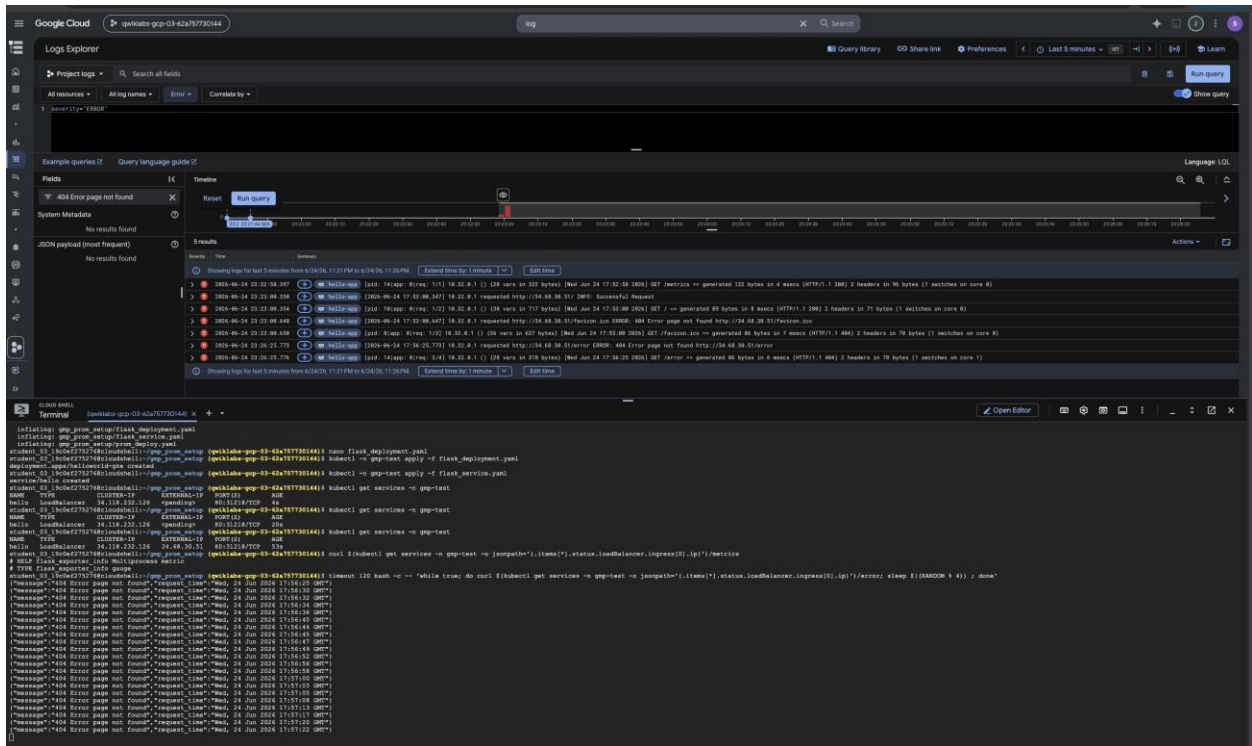
```
curl -X GET http://10.1.1.1:8080
{"message": "Hello World!", "timestamp": "2024-06-24 17:53:00 GMT"}
```

Step 9 – Generate Application Logs

Description

Normal traffic and HTTP 404 errors were generated for monitoring.

Insert Screenshot



Step 10 – Create Log-Based Metric

Description

A custom log-based metric was created to count HTTP 404 errors.

Step 11 – Configure Alert Policy

Description

A Cloud Monitoring alert policy was configured using the custom metric.

Insert Screenshot

×

Create log-based metric

Send feedback

Configure the settings below to define and create a log-based metric.

Metric Type

Counter

Counts the number of log entries matching a given filter

[Learn more](#)

Distribution

Collects numeric data from log entries matching a given filter

[Learn more](#)

Details

Log-based metric name *

hello-app-error

Description

Enter a description for this metric (optional)

Units

The units of measurement that apply to this metric (for example, bytes or seconds). For counter metrics, leave this blank or insert the digit "1". For distribution metrics, you can optionally enter units, such as "s", "ms", etc. [Learn more](#)

Filter selection

Preview logs

Define your log-based metric

Select project or log bucket

Project logs

Select whether your log-based metric examines (or counts) log entries that are routed by your project, or only those log entries in a log bucket.

Build filter *

Press Alt+F1 for accessibility options.

Query language

1 severity=ERROR

2 resource.labels.container_name="hello-app"

3 textPayload:"ERROR: 404 Error page not found"

Labels

Labels allow log-based metrics to contain multiple time series [Learn more](#)

+ Add label

Create metric

Cancel

Step 12 – Configure Notification Channel

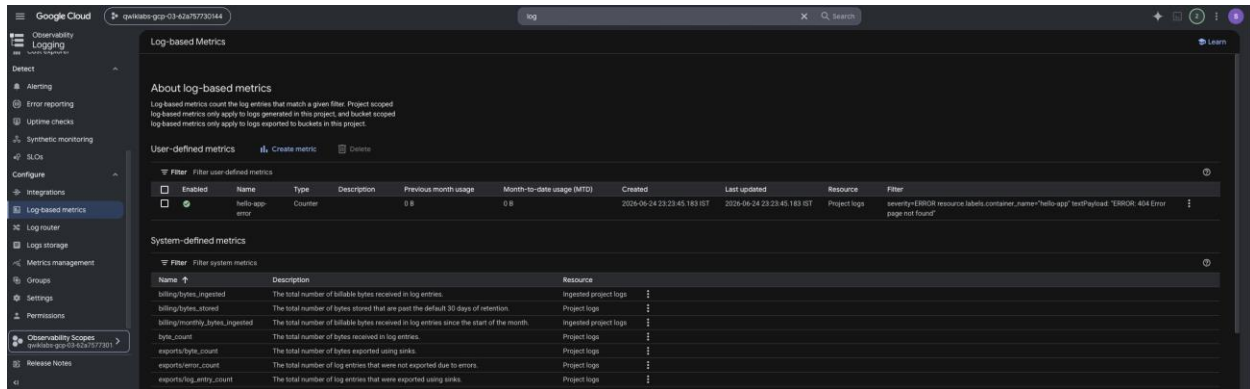
Description

Email notifications were configured for monitoring alerts.

Step 14 – Verify Logs Explorer

Description

Application logs were validated using Logs Explorer.



8. Results

The project successfully demonstrated:

- Kubernetes deployment
- Docker image management
- Artifact Registry integration
- Cloud Logging
- Log-Based Metrics
- Monitoring dashboards
- Alert Policy creation
- Notification Channel configuration
- Incident monitoring

9. Challenges Faced

- Configuring Kubernetes resources.
- Waiting for LoadBalancer IP allocation.
- Generating application errors.
- Configuring log filters.
- Validating monitoring alerts.

- Understanding Cloud Monitoring metrics.

10. Learning Outcomes

This project provided hands-on experience in:

- Google Kubernetes Engine
- Artifact Registry
- Kubernetes Deployments
- Cloud Logging
- Cloud Monitoring
- Alert Policies
- Log-Based Metrics
- Notification Channels
- Cloud Observability
- Incident Response

11. Conclusion

This project successfully implemented a complete cloud observability solution using Google Cloud Platform. Containerized applications were deployed on Google Kubernetes Engine, application logs were collected through Cloud Logging, custom log-based metrics were generated, and automated alert policies were configured to detect application issues. The project strengthened practical knowledge of Kubernetes, monitoring, logging, and cloud operations while demonstrating production-ready monitoring workflows commonly used in enterprise environments.